

**Rapport de stage : Mémoire et continuité du dialogue pour
assistant vocal IA – ISIA Lab, UMONS**

COTTEL Thibault

Cycle ingénieur 5A : Audiovisuel et Multimédia

Stage du 16 Mars 2026 au 17 Août 2026

Tuteur industriel : Omar Seddati

Tuteur enseignant : Alexis Heloir

SOMMAIRE

I. Formalités.....	5
1.1 Engagement de non-plagiats.....	5
1.2 Remerciements	5
1.3 Introduction aux sigles et abréviations	6
II. Introduction.....	7
2.1 Résumé	7
2.2 Présentation du laboratoire	7
2.3 Présentation du projet VisIA et missions du stage.....	7
III. Cadrage technique et état de l'art	9
3.1 Outils et environnement de développement	9
3.2 État de l'art commercial.....	10
3.2.1 Applications mobiles d'assistance visuelle	10
3.2.2 Lunettes intelligentes et dispositifs physiques embarqués.....	10
3.2.3 Constat de l'état de l'art commercial	11
3.3 Évaluation et limites des framework existants.....	12
3.4 État de l'art des architectures mémorielles.....	13
3.4.1 Gestion de la chronologie : Temporal Semantic Memory (TSM)	13
3.4.2 Structuration et enrichissement : Metadata-as-Text (MaT).....	14
3.4.3 Consolidation et graphes de connaissances : FastInsight.....	15
3.4.4 Mécanismes d'oubli actif et filtrage : FadeMem et MemCtrl	15
3.5 Analyse et définition des scénarios utilisateurs.....	16
3.5.1 Identification et priorisation des usecases	16
3.5.2 Mode réactif vs mode proactif	16
IV. Architecture et Implémentation technique.....	18
4.1 Introduction au pipeline RAG local	18
4.2 Évolution du projet : de la V1 à la V5	19
4.2.1 RAG sémantique naïf (V1).....	19
4.2.2 Approche par fichiers (V2)	19
4.2.3 Centralisation sous ChromaDB et décroissance temporelle (V3).....	19
4.2.4 Intégration du graphe sémantique (V4)	20
4.2.5 Architecture hybride et optimisation du routage (V5).....	21
4.3 Spécifications des Modules	22
4.3.1 Classification et routage d'intentions (Intention Router).....	22

4.3.2 Gestion des coréférences et traitement de la syntaxe française	23
V. Validation et analyse des résultats	25
5.1 Protocoles de test et métriques d'évaluation	25
5.1.1 Analyse des benchmarks existants	25
5.1.2 Enseignements tirés des benchmarks existants	25
5.2 Analyse globale des résultats	26
5.2.1 Optimisations de l'architecture et gains de performance.....	26
5.2.2 Bilan final des performances de la V5	28
VI. Conclusion	30
VII. Annexes.....	31
7.1 Annexe A : Liens et références des outils et frameworks :	31
7.2 Annexe B : Veille technologique et solutions existantes sur le marché	32

Liste des Figures

Figure 1 : Schéma de la démarche itérative utilisée	8
Figure 2 : Aperçu de l'interface de démonstration VisIA V3 sous Gradio.....	9
Figure 3 : Architecture du fonctionnement du Temporal Semantic Memory, schéma issu de l'article "Beyond Dialogue Time : Temporal Semantic Memory" (Su et al, 2026)	14
Figure 4: Schéma de la stratégie MaT, adapté de l'article "Utilizing Metadata for Better Retrieval-Augmented Generation"	15
Figure 5 : Extrait du tableau des scénarios utilisateurs	16
Figure 6 : Schéma du flux de l'architecture du système de mémoire VisIA V3	18
Figure 7 : Courbe de l'oubli de Hermann Ebbinghaus	20
Figure 8 : Schéma du principe de rupture d'arc lors d'un changement dans la scène.....	21
Figure 9 : Exemple de routage de l'Intention Router.....	23
Figure 10 : Graphe de précision de la résolution de coréférences par catégorie de test	24
Figure 11 : Matrice de Confusion du Routeur d'Intentions (V3).....	26
Figure 12 : Évolution de la distance vectorielle ajustée en fonction du temps.....	27
Figure 13 : Comparaison des gains de latence (avec et sans optimisation de préchargement)	27
Figure 14 : Taux de succès du routeur par intention (V5).....	28
Figure 15 : Distribution de la latence des requêtes (V5).....	29
Figure 16 : Diagramme du temps de réponse du système en fonction de l'intention (V5)	29
Figure 17 : Boîtier OrCam MyEye fixé sur monture	33
Figure 18 : Lunettes connectées Envision glasses (Ally)	33
Figure 19 : Lunettes connectées Meta Rayban	33
Figure 20 : Boîtier AngelEye	33
Figure 21 : Boîtier Sidekick fixé sur monture	33
Figure 22 : Lunettes connectées Agiga Echovision	33

I. FORMALITES

1.1 Engagement de non-plagiats

Je soussigné **COTTEL Thibault**, étudiant en Spécialité Ingénieur **Audiovisuel et Multimédia** (AVM) déclare être pleinement conscient que le plagiat d'un document ou d'une partie de document publié sur toutes les formes existantes de support, y compris sur Internet, constitue une violation des droits d'auteur ainsi qu'une fraude caractérisée. En conséquence, je m'engage à citer explicitement, à chaque fois que j'en fais usage, toutes les sources que j'ai utilisées pour écrire ce rapport.

Fait à Mons, le 06/08/2026



Signature :

1.2 Remerciements

Avant tout développement, il apparait opportun de commencer ce rapport de stage par des remerciements.

Je voudrais dans un premier temps remercier l'ensemble de l'équipe de l'ISIA Lab pour son accueil, sa confiance et sa disponibilité.

Je tiens plus particulièrement à remercier :

- Omar SEDDATI, tuteur de stage. Son accueil, son expertise, ainsi que les discussions que j'ai pu avoir avec lui qui m'ont beaucoup apporté pendant toute la durée de mon stage.

Je tiens également à remercier l'ensemble de l'équipe pédagogique et administrative de l'université de Valenciennes pour l'accompagnement effectué au cours de ce stage.

- Alexis HELOIR, tuteur enseignant à l'INSA Hauts de France, ainsi qu'à l'équipe pédagogique, pour leur suivi et leur écoute au long de ce stage, et plus largement au long de mon cursus dans la formation.
- L'ensemble de l'équipe de l'ISIA Lab, pour leur accueil et leur sympathie au quotidien.

Enfin, je remercie toutes les personnes qui m'ont aidé de près ou de loin lors de cette période.

1.3 Introduction aux sigles et abréviations

Ce rapport contient un certain nombre de sigles et abréviations. Afin de faciliter la lecture du document, une liste comprenant l'intégralité des sigles utilisés est disponible ci-dessous :

- API : Application Programming Interface
- AVM : Audiovisuel et Multimédia
- CPU : Central processing Unit
- GPU : Graphic Processing Unit
- INSA : Institut National des Sciences Appliquées
- IHM : Interaction Homme-Machine
- ISIA LAB : Laboratoire d'Information, Signal et Intelligence Artificielle
- JSON : JavaScript Object Notation
- LLM : Large Language Model. Modèle d'intelligence artificielle entraîné sur de grands volumes de données capable de reconnaître et de générer du texte.
- LML : Long-term Memory Layer
- MaT : Metadata-as-Text
- MIT : Massachusetts Institute of Technology
- NLP : Natural Language Processing
- OCR : Optical Character Recognition
- RAG : Retrieval-Augmented-Generation. Technique d'intelligence artificielle qui consiste à connecter un modèle de langage à des sources de données externes.
- RL : Reinforcement Learning
- SDK : Software Development Kit
- SML : Short-term Memory Layer
- STT : Speech-To-Text
- TRAIL : TRusted AI Labs
- TSM : Temporal Semantic Memory
- TTS : Text-To-Speech
- UI : User Interface
- UMONS : Université de Mons
- VLM : Visual Language Model
- VQA : Visual Question Answering
- VRAM : Video Random Access Memory

II. INTRODUCTION

2.1 Résumé

Ce stage de fin d'étude s'est déroulé au sein du laboratoire d'Information, Signal et Intelligence Artificielle (ISIA Lab) de la faculté Polytechnique de l'Université de Mons (UMONS), en Belgique. Il constitue une étape clé de mon cursus au sein de la spécialité Audiovisuel et Multimédia (AVM) de l'Institut National des Sciences Appliquées (INSA) Hauts-de-France. Mes travaux s'inscrivent dans le cadre du projet VisIA, un assistant intelligent conçu pour accompagner au quotidien les personnes souffrant de déficience visuelle. Aujourd'hui, la plupart des systèmes d'aide visuelle fonctionnent de manière instantanée, c'est-à-dire qu'ils décrivent ce qui se trouve devant l'utilisateur à un instant précis, mais oublient l'information dès que la caméra change de champ. L'objectif de mon stage a donc été de rechercher, concevoir et valider une architecture mémorielle persistante. Cette architecture servira à l'assistant final et lui permettra de stocker, trier et retrouver les informations importantes au fil du temps.

Afin de présenter au mieux la démarche scientifique que j'ai menée ces derniers mois, le rapport expose d'abord le cadrage technique et l'état de l'art du projet, puis détaille l'architecture développée ainsi que les résultats obtenus.

2.2 Présentation du laboratoire

Le laboratoire de recherche ISIA fait partie intégrante de la Faculté Polytechnique de l'Université de Mons, en Belgique. Il constitue l'un des acteurs majeurs de l'institut de recherche NUMEDIART, une structure interdisciplinaire consacrée aux technologies numériques créatives et à l'ingénierie des médias. Le laboratoire s'inscrit également dans des réseaux d'excellence académiques et industriels d'envergure régionale et européenne, figurant notamment parmi les membres fondateurs de l'institut TRAIL (TRusted AI Labs), dédié au développement d'une intelligence artificielle digne de confiance. L'ISIA Lab bénéficie d'une reconnaissance internationale dans les domaines du traitement du signal, de la vision par ordinateur et de l'intelligence artificielle multimodale. Les travaux menés au sein de l'unité reposent sur l'analyse, la synthèse, la reconnaissance et la cartographie d'une grande diversité de flux de données, incluant la parole, l'audio, les images fixes, le flux vidéo, les données oculométriques, les gestes corporels et les biosignaux. Grâce à la maîtrise des architectures d'apprentissage profond (Deep Learning), telles que les réseaux convolutifs, les réseaux génératifs ou les modèles de fondation basés sur les transformeurs, le laboratoire développe des solutions algorithmiques capables d'interpréter des scènes complexes et de concevoir des interfaces d'interaction homme-machine intuitives et réactives.

2.3 Présentation du projet VisIA et missions du stage

Ce stage s'inscrit au cœur du projet VisIA, une initiative de recherche et de développement menée au sein du laboratoire ISIA visant à concevoir un compagnon virtuel intelligent pour les personnes aveugles et malvoyantes. L'ambition globale de VisIA est d'apporter une aide concrète aux tâches de la vie quotidienne grâce aux dernières avancées de l'intelligence artificielle. L'agent final, sous la forme de lunettes connectées, intègre une

interface de commande vocale permettant à l'utilisateur d'interagir naturellement avec le système pour effectuer des requêtes variées, allant de la recherche d'informations sur Internet à la lecture automatique d'enseignes ou de panneaux de signalisation dans la rue, en passant par la détection et la localisation d'objets personnels égarés dans un espace domestique.

La mission qui m'a été confiée au cours de ce projet touche à l'un des défis majeurs en intelligence artificielle : la conception, la persistance et la contextualisation de la mémoire des agents autonomes.

Afin de mener à bien cette mission, la méthode de travail utilisée est celle d'une démarche agile par étapes de sprints successifs. Chaque cycle court comprenait des phases de recherche, de définition, d'implémentation et de validation, permettant le bon avancement du projet. De manière globale, le travail s'est organisé en trois grandes phases :

- Phase d'étude et de prototypage : Cette première étape concerne l'analyse de l'état de l'art des systèmes existants, l'analyse des outils et framework existants, ainsi que la définition des scénarios d'usages (use cases).
- Phase de conception et d'implémentation du système : Cette seconde phase est caractérisée par la conception de l'architecture du système ainsi que le développement du pipeline logiciel.
- Phase de tests et optimisation structurelle : Évaluation du système via des campagnes de tests (analyse de la cohérence des réponses, analyse de la latence, traitement des cas limites).



Figure 1 : Schéma de la démarche itérative utilisée

III. CADRAGE TECHNIQUE ET ETAT DE L'ART

3.1 Outils et environnement de développement

La mise en œuvre de l'architecture du système a été réalisée à l'aide d'outils logiciels sélectionnés pour leur robustesse, leur gratuité, ou encore car je les avais déjà utilisés par le passé.

- Langage de programmation : L'intégralité du pipeline logiciel est développée en Python. Ce choix est justifié par le nombre important de bibliothèques et de projets d'intelligence artificielle dans ce langage.
- Gestion de version : Git et la plateforme GitHub ont été utilisés pour la traçabilité du code écrit tout au long du stage. Cela a permis de conserver un historique précis des modifications.
- Serveur d'inférence locale : L'outil open-source Ollama a été utilisé comme serveur d'inférence local. Cela a permis l'utilisation de modèles tels que Gemma 3 (4B) et Llama 3.2 1B, sans dépendance à des API cloud. Autrement dit, cela garantit le respect de la confidentialité des données de l'utilisateur.
- Base de données vectorielle : Le stockage et la persistance de la mémoire à long terme est réalisé grâce à ChromaDB combiné au framework Mem0. ChromaDB permet de transformer des textes en vecteurs mathématiques (embeddings) afin de retrouver les souvenirs par sens (recherche sémantique). Mem0, quant à lui, est utilisé pour la structuration et la mise à jour des profils et des habitudes de l'utilisateur.
- Interface Homme-Machine (IHM) : La bibliothèque open-source Python Gradio a été choisie pour réaliser l'interface web. Ce choix s'appuie sur la simplicité et la gratuité de l'outil, permettant de développer des interfaces web interactives pour des modèles d'intelligence artificielle.
- Environnement de prototypage : Si les premiers scripts ont été écrits sous Visual Studio Code, l'environnement principal utilisé est rapidement devenu Google Colab. Ce choix s'explique par la possibilité d'utiliser un processeur graphique (GPU) plus puissant que celui de mon ordinateur personnel, accélérant ainsi le temps de calcul des modèles.

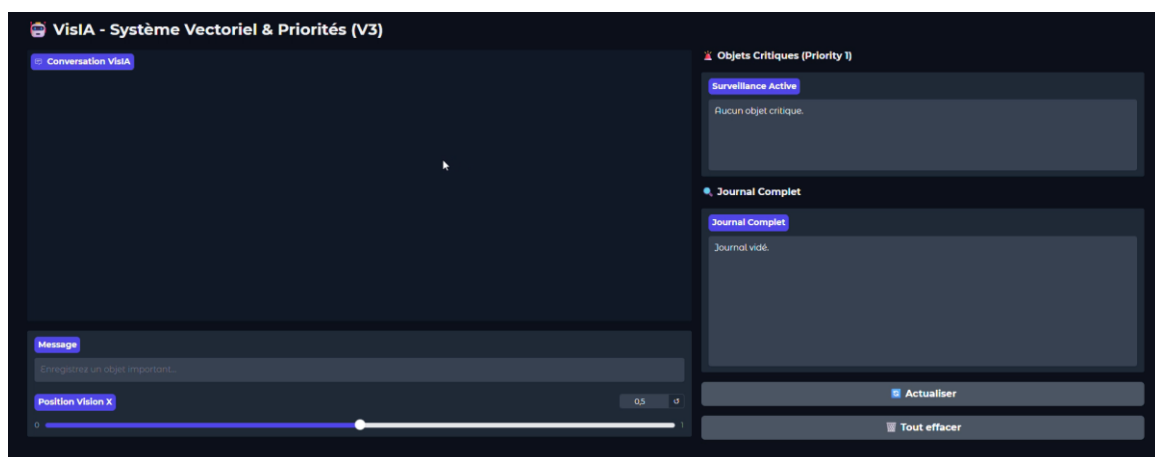


Figure 2 : Aperçu de l'interface de démonstration VisIA V3 sous Gradio

3.2 État de l'art commercial

Afin d'avoir un aperçu global du projet VisIA par rapport aux technologies existantes, une veille technologique a été réalisée sur les solutions d'assistance visuelle disponibles sur le marché. L'étude a porté sur deux catégories : les applications mobiles d'une part, et les lunettes intelligentes ou dispositifs physiques de l'autre.

3.2.1 Applications mobiles d'assistance visuelle

Il existe de nombreuses solutions logicielles sur smartphone qui ont notamment l'avantage d'être facilement accessibles pour les utilisateurs malvoyants.

- **Be My Eyes** : Cette application gratuite est l'un des leaders du secteur. Historiquement, cette application était basée sur du crowdsourcing permettant à des volontaires d'aider les utilisateurs. Aujourd'hui, elle intègre un outil d'intelligence artificielle : Be My AI. Ce système s'appuie sur un modèle de langage visuel (VLM) via les API cloud de GPT-4o pour générer des descriptions de scènes. Malgré sa capacité à analyser des scènes complexes, le traitement à distance introduit une latence sur les réponses.
- **Microsoft Seeing AI** : Cette application est gratuite, elle se distingue par l'utilisation de l'edge computing (plusieurs modèles s'exécutent localement sur le téléphone de l'utilisateur, cela permet d'offrir une réponse instantanée). Parmi les fonctionnalités proposées par l'application, on retrouve la lecture de document ou encore l'analyse de scènes. À noter que dans le cas de scènes complexes, elle bascule sur le cloud de Microsoft. L'application permet également d'estimer la lumière ambiante ou encore de reconnaître des visages avec des profils enregistrés.
- **Google Lookout** : Lookout fonctionne par sessions indépendantes et utilise des technologies de réponse visuelle aux questions (VQA). Elle intègre une fonctionnalité de recherche ciblée pour localiser un objet précis dans l'espace.
- **Oorion** : Application française spécialisée exclusivement dans l'identification, la localisation et le guidage précis de l'utilisateur vers des objets ou des textes précis.

3.2.2 Lunettes intelligentes et dispositifs physiques embarqués

Sur le marché, il existe des dispositifs embarqués qui ont l'avantage principal d'une utilisation mains libres, cruciale pour les personnes non-voyantes. Les dispositifs présentés dans cette section sont illustrés en [Annexe B, figures 17 à 22](#).

- **OrCam MyEye** : OrCam MyEye est un boîtier physique qui se fixe sur les branches de lunettes ordinaires. Il fonctionne 100% hors ligne sans nécessiter de connexion Internet. Cette solution permet de faire de la lecture de texte ou de l'identification de visages mémorisés localement. Ses principales limites sont le coût élevé ainsi que l'absence de compréhension sémantique de l'environnement par le système.
- **Envision glasses (Ally)** : Envision combine une paire de lunettes connectées et une application mobile. Elle propose une reconnaissance optique de caractères (OCR) ainsi qu'une fonctionnalité de reconnaissance faciale. Ally propose un outil de dialogue conversationnel disposant d'une mémoire à très court terme (un buffer enregistrant les cinq dernières minutes). Quelques limites sont à souligner comme l'autonomie de la batterie ou encore le coût important.

- **Meta Ray-ban** : Les lunettes Meta Ray-Ban sont les plus connues parmi les lunettes connectées sur le marché. Ces lunettes utilisent l'intelligence artificielle multimodale Meta AI with Vision. Elles permettent d'analyser des images instantanées à un prix grand public et peuvent donc tout à fait être utilisées par des personnes déficientes visuelles. En revanche cette solution ne conserve pas de trace des requêtes précédentes.
- **Solutions alternatives (Angel Eye, Sidekick, Echovision)** : D'autres dispositifs explorent des approches variées, comme Angel Eye qui est un boîtier à clipser sur les lunettes et qui permet de faire de la lecture de documents. Sidekick développe une plateforme open-source avec une volonté d'intégrer une persistance mémorielle, le produit est en cours de développement. Enfin, Agiga Echovision est une solution qui a la particularité d'utiliser des boutons physiques plutôt qu'une commande vocale pour demander à décrire la scène.

3.2.3 Constat de l'état de l'art commercial

L'analyse comparative des solutions proposées sur le marché aujourd'hui met en évidence une lacune technologique majeure : la quasi-totalité des produits actuels repose sur une architecture dite « stateless », c'est-à-dire qu'ils reposent sur de la vision instantanée mais ne conservent aucun historique des requêtes passées. Cette absence de persistance à long terme s'explique par plusieurs verrous technologiques :

- Indexer et stocker des bases de données vectorielles pour des millions d'utilisateurs en continu représente un coût d'infrastructure serveur important.
- Enregistrer en continu la vision d'un individu soulève des questions d'éthique et de respect de la vie privée.
- Certifier à 100% qu'un objet est le même en faisant varier l'angle de perception, la luminosité, ou le flou est un défi technologique pour lequel le taux d'erreur est trop élevé aujourd'hui.

Solution	Support	Mémoire & Persistance	Points Forts	Coût Approximatif
Be My Eyes	App	Stateless	Crowdsourcing + Puissance GPT-4o	Gratuit
Microsoft Seeing AI	App	Stateless	Rapidité Edge computing	Gratuit
Google Lookout	App	Stateless	Module « find » pour la localisation d'objets	Gratuit
Oorion	App	Stateless	Guidage audio vers objet cible	Gratuit
OrCam MyEye	Boîtier dédié	Mémoire de reconnaissance brute (visages et produits enregistrés en local. Pas de conversations stockées)	100% Hors ligne et très rapide	2200€-4800€
Envision glasses	Lunettes	Mémoire court-terme (buffer de 5minutes)	Mains libres + OCR haute qualité	2000€-3600€
Meta Ray-Ban	Lunettes	Stateless	Mains libres + Discret + Écosystème Meta	330€-500€

Tableau 1 : Tableau comparatif des solutions d'assistance visuelle du marché

3.3 Évaluation et limites des framework existants

Afin de découvrir les mécanismes logiciels déjà disponibles, mon travail a commencé par une étude comparative des frameworks de mémoire existant. L'objectif de cette évaluation était d'abord d'identifier si une solution open-source existante pouvait être directement appliquée à mes travaux, tout en respectant plusieurs critères : la gestion de la mémoire à court et long terme, une licence permissive Apache 2.0 ou MIT (Massachusetts Institute of Technology), et une légèreté algorithmique compatible avec un système embarqué.

Au total, dix solutions logicielles ont été répertoriées et analysées selon leur maturité (nombre d'étoiles sur GitHub), la maintenance du projet, la capacité à gérer simultanément la mémoire à court terme et la mémoire à long terme. Voici une liste des solutions étudiées : Mem0, OpenMemory, Cognee, Letta (anciennement MemGPT), Zep, Langchain, Motorhead,

NeuroMemory, CrewAI, LlamaIndex. Parmi elles, trois solutions principales se sont distinguées :

- LangChain : Ce framework est le plus populaire pour développer des applications basées sur des LLM, il dispose d'une maintenance active et propose des modules pour la mémoire courte ainsi que pour la mémoire longue. Cependant l'architecture est très lourde pour notre cas d'usage.
- Letta : Cette solution est très robuste, elle reprend le principe de fonctionnement d'un système d'exploitation avec une mémoire vive pour l'accès rapide et une mémoire d'archivage pour le stockage de masse. Cependant, la configuration de Letta s'avère trop complexe pour l'infrastructure cible, c'est pourquoi cette solution a été écartée.
- Mem0 : Mem0 a d'abord été identifié comme la solution la plus équilibrée, avec une approche légère et un SDK (Software Development Kit) Python simple à intégrer. Elle gère la mémoire à plusieurs niveaux. Mem0 a donc été choisi et testé. Ces premiers tests ont révélé des difficultés d'adaptation avec un modèle léger tel que Gemma 3 (4B). C'est pourquoi Mem0 a été écarté dans un premier temps.

L'analyse de ces frameworks mémoire et les différents échecs lors des expérimentations ont montré que ces outils « généralistes » ne sont pas directement adaptés aux contraintes d'un petit modèle. Ainsi la décision a été prise de concevoir une architecture sur-mesure, connectant Gemma 3 (4B) à la base vectorielle ChromaDB, dans le cadre de ce stage pour le projet VisIA. Par la suite du projet, et face au besoin d'une brique dédiée aux préférences et aux routines de l'utilisateur, le framework Mem0 a finalement été réintégré et adapté à partir de la version 5 (V5).

3.4 État de l'art des architectures mémorielles

Au cours de mon stage j'ai eu l'occasion d'étudier plusieurs articles scientifiques récents. L'objectif de cette démarche était de me renseigner sur les théories et concepts émergents en termes de conception de systèmes mémoriel. L'intégralité des articles mentionnés sont disponibles en annexe ([Annexe A > Références et articles scientifiques étudiées](#)). Quatre concepts algorithmiques clés ont inspiré mon système.

3.4.1 Gestion de la chronologie : Temporal Semantic Memory (TSM)

Les systèmes de RAG traditionnels indexent les souvenirs à l'aide de l'horodatage au moment de la discussion (lorsqu'une phrase est prononcée, l'heure exacte est extraite et assignée au souvenir). L'article "Beyond Dialogue Time : Temporal Semantic Memory" (Su et al, 2026) explique que cela provoque une inexactitude dans les réponses car le système a tendance à créer des décalages et des confusions entre présent, passé et futur.

Afin d'y remédier, le concept de Temporal Semantic Memory (TSM) est introduit par l'article. Cela consiste à passer à une frise chronologique sémantique. L'architecture est capable de reconnaître et d'isoler des expressions temporelles dans la requête de l'utilisateur (ex : « ce matin », « lundi dernier », ...) pour les convertir en dates. Ensuite, lors de la phase de récupération, un filtrage et un reclassement sont effectués, un bonus de score est appliqué aux souvenirs alignés avec la fenêtre temporelle visée par la question.

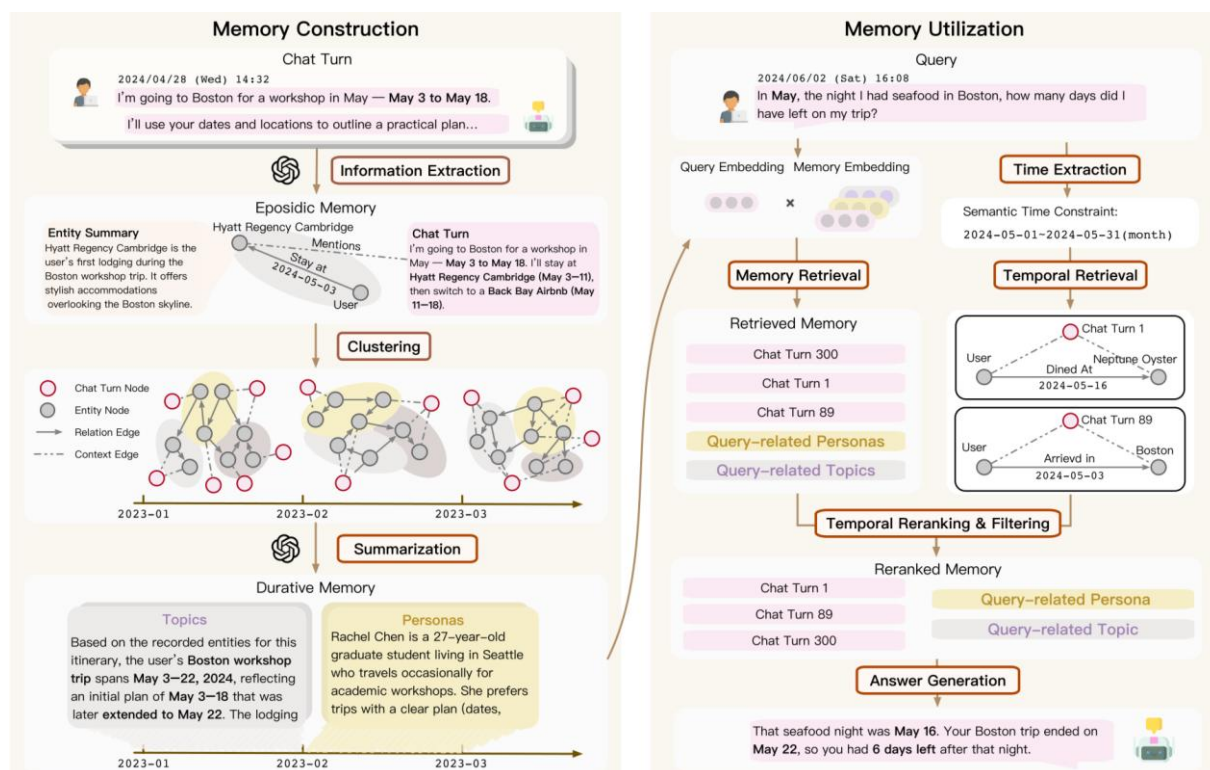


Figure 3 : Architecture du fonctionnement du Temporal Semantic Memory, schéma issu de l'article "Beyond Dialogue Time : Temporal Semantic Memory" (Su et al, 2026)

3.4.2 Structuration et enrichissement : Metadata-as-Text (MaT)

L'article "Utilizing Metadata for Better Retrieval-Augmented Generation" (Yousuf et al. 2026) souligne un problème concernant les petits modèles d'IA. Si plusieurs souvenirs se ressemblent sémantiquement, alors le modèle est incapable de déterminer de manière fiable la donnée la plus pertinente ou la plus récente. La solution proposée à ce problème est nommée Metadata-as-Text (MaT). Au lieu d'encoder le texte brut seul, cette solution propose de joindre les métadonnées liées au contexte (nom de l'objet, horodatage, position) au texte pour former une seule phrase. Cela se produit au moment de la sauvegarde, ainsi les calculs d'embeddings ne sont réalisés qu'une seule fois.

Stratégie utilisée	Flux de donnée envoyé à la base vectorielle
Sans MaT	["Le sac est posé sur la table"]
Avec MaT	["Objet: Sac Date: 31 Août Heure: 16h15 Position: Gauche Texte: Le sac est posé sur la table"]

Tableau 2: Tableau d'exemple d'application de la solution MaT pour le projet VisIA

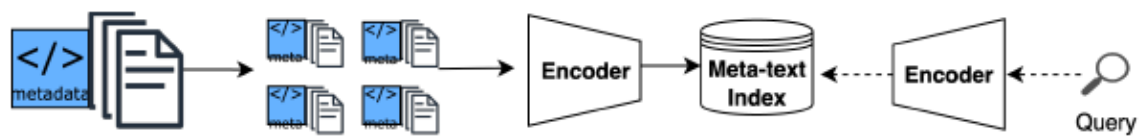


Figure 4: Schéma de la stratégie MaT, adapté de l'article "Utilizing Metadata for Better Retrieval-Augmented Generation"

3.4.3 Consolidation et graphes de connaissances : FastInsight

Le troisième concept retenu de ma veille technologique est issu de l'article "FastInsight: Fast and Insightful Retrieval via Fusion Operators for Graph RAG" (An et al, 2026). Cette étude aborde la problématique du coût de calcul et de la latence des systèmes mémoriels avancés. Les approches de recherche actuelles s'appuient sur des processus lourds qui sollicitent un LLM à chaque étape, ce qui a pour conséquence de saturer la mémoire et d'augmenter la latence des réponses. Les auteurs proposent dans cet article de remplacer le raisonnement lourd par des opérations mathématiques appliquées sur un graphe sémantique. FastInsight introduit deux nouveaux opérateurs :

- Graph-based Reranker (Granker) : Cet opérateur traite les scores initiaux de la recherche textuelle comme des signaux bruités. Il ajuste le classement d'un souvenir en exploitant la structure du graphe sémantique. Par exemple, si l'objet « sac » est jugé très pertinent par rapport à la requête, tous les objets connectés ou enregistrés à ses côtés (comme des « clés ») recevront un bonus de score.
- Semantic-Topological eXpansion (STeX) : Cet algorithme identifie les meilleurs candidats mémoriels en combinant simultanément la similarité vectorielle (sens des mots) et la proximité topologique au sein du graphe.

3.4.4 Mécanismes d'oubli actif et filtrage : FadeMem et MemCtrl

La gestion de l'encombrement de la mémoire et le traitement des flux de données en temps réel constituent le dernier verrou majeur de l'état de l'art. Contrairement aux systèmes RAG classiques conçus pour de l'analyse statique, un assistant de navigation comme VisIA fait face à un flux continu d'observations visuelles et textuelles. Conserver l'intégralité de ces interactions sans filtrage entraînerait inévitablement une saturation de la fenêtre de contexte. Cela provoquerait également une hausse de la latence d'inférence ainsi que l'apparition de contradictions sémantiques. Pour y remédier, l'article "FadeMem: Biologically Inspired Forgetting for Efficient Agent Memory" (Wei et al., 2026) introduit une architecture mémorielle inspirée du cerveau humain, intégrant des mécanismes d'oubli actif. FadeMem s'appuie sur un fonctionnement à deux couches :

- La couche à court terme (SML – Short-term Memory Layer) : Destinée aux souvenirs de faible importance, programmés pour s'effacer rapidement.
- La couche à long terme (LML – Long-term Memory Layer) : Réservée aux informations importantes ainsi qu'à celles fréquemment utilisées.

Le framework évalue la relation entre les nouveaux souvenirs et l'historique existant pour déclencher soit une fusion des doublons, soit une suppression des fichiers obsolètes. Dans la même idée, l'article "MemCtrl: Using MLLMs as Active Memory Controllers on

Embodied Agents" (Dorbala et Dinesh, 2026) développe un système décidant en temps réel si une information doit être conservée (Retain), mise à jour (Update) ou rejetée (Discard). Bien que l'article MemCtrl préconise l'utilisation de l'apprentissage par renforcement (RL) pour entraîner le système, ce n'est pas une approche que j'ai pu tester au cours de mon stage.

3.5 Analyse et définition des scénarios utilisateurs

Une phase essentielle du début de stage a été l'analyse du besoin afin de définir et prioriser les différents scénarios d'utilisation de VisIA pour une personne malvoyante dans son quotidien. L'objectif était double : identifier précisément les attentes des utilisateurs et donner une direction pour le développement du système. Plusieurs moyens ont été utilisés pour définir ces scénarios, tout d'abord j'ai pu avoir des discussions avec mon tuteur, j'ai aussi réfléchi de mon côté, ensuite j'ai également discuté avec Gemini qui m'a proposé des scénarios auxquels je n'avais pas pensé, et finalement j'ai été chercher des informations sur des forums spécialisés (ex. le reddit : r/Blind).

3.5.1 Identification et priorisation des usecases

Lors de la classification des scénarios, le choix a été fait de les séparer en trois catégories :

- **Priorité 1 (Critique) :** Gestion des objets critiques (santé et sécurité de l'utilisateur)
 - Exemples de scénarios : Localiser une boîte de médicaments, avertir d'un obstacle au sol, avertir d'un objet dangereux (couteau par exemple).
- **Priorité 2 (Haute) :** Gestion des objets du quotidien en mouvement fréquent
 - Exemples de scénarios : Retrouver un téléphone, un portefeuille ou une paire de lunettes.
- **Priorité 3 (Moyenne) :** Gestion de l'environnement stable
 - Exemples de scénarios : Identifier la position d'un objet fixe (chaise, bouteille d'eau), lecture de texte (OCR), description globale de l'environnement.

ID	Nom du scénario	Catégorie	Objectif testé	Priorité	Nb Inputs	Nb Questions	Description
iC-001	Localisation du café	Environnement personnel	Retrouver un objet dans la cuisine	Haute	3	2	Retrouver un objet dans la cuisine
iC-002	Disposition du salon	Environnement personnel	Décrire l'agencement du salon	Haute	3	2	Décrire l'agencement du salon
iC-003	Chambre à coucher	Environnement personnel	Retrouver des objets dans la chambre	Haute	3	2	Retrouver des objets dans la chambre
iC-004	Salle de bain	Environnement personnel	Localiser des produits d'hygiène	Moyenne	3	2	Localiser des produits d'hygiène
iC-005	Entrée et couloir	Environnement personnel	Retrouver affaires dans l'entrée	Moyenne	3	2	Retrouver affaires dans l'entrée
iC-006	Garage et rangement	Environnement personnel	Retrouver des outils	Basse	3	2	Retrouver des outils
iC-007	Bureau à domicile	Environnement personnel	Retrouver des documents et matériel	Haute	3	2	Retrouver des documents et matériel
iC-008	Placard à provisions	Environnement personnel	Retrouver des aliments	Moyenne	3	2	Retrouver des aliments
iC-009	Frigo organisation	Environnement personnel	Localiser des aliments au frigo	Moyenne	3	2	Localiser des aliments au frigo
iC-010	Buanderie	Environnement personnel	Retrouver des produits ménagers	Basse	3	2	Retrouver des produits ménagers
iC-011	Balcon et terrasse	Environnement personnel	Localiser des objets extérieurs	Basse	3	2	Localiser des objets extérieurs
iC-012	Cave et sous-sol	Environnement personnel	Localiser des affaires en cave	Basse	3	2	Localiser des affaires en cave
iC-013	Chambre d'amis	Environnement personnel	Préparer la chambre d'amis	Basse	3	2	Préparer la chambre d'amis
iC-014	Coin repas	Environnement personnel	Localiser la vaisselle	Moyenne	3	2	Localiser la vaisselle
iC-015	Dressing et vêtements	Environnement personnel	Retrouver des vêtements	Haute	3	2	Retrouver des vêtements
iC-016	Cuisine — épiceries	Environnement personnel	Retrouver les épiceries	Moyenne	3	2	Retrouver les épiceries
iC-017	Espace enfants	Environnement personnel	Retrouver jouets et affaires d'enfant	Moyenne	3	2	Retrouver jouets et affaires d'enfant
iC-018	Pièce multimédia	Environnement personnel	Retrouver équipement multimédia	Basse	3	2	Retrouver équipement multimédia
iC-019	Armoire à pharmacie	Environnement personnel	Retrouver des médicaments spécifiques	Critique	3	2	Retrouver des médicaments spécifiques
iC-020	Véranda	Environnement personnel	Localiser affaires dans la véranda	Basse	3	2	Localiser affaires dans la véranda
iC-021	Routine médicaments matin	Habitudes quotidiennes	Restituer routine médicamenteuse	Critique	3	2	Restituer routine médicamenteuse
iC-022	Routine du soir	Habitudes quotidiennes	Restituer routine du coucher	Haute	3	2	Restituer routine du coucher
iC-023	Petit-déjeuner	Habitudes quotidiennes	Rappeler les habitudes alimentaires du matin	Moyenne	3	2	Rappeler les habitudes alimentaires du matin

Figure 5 : Extrait du tableau des scénarios utilisateurs

3.5.2 Mode réactif vs mode proactif

L'analyse des use-cases a montré qu'il existe deux types de scénarios distincts. En effet, le système peut utiliser un mode réactif pour lequel il n'agit que sur demande de l'utilisateur (via une requête explicite), ou bien un mode proactif, où l'assistant analyse le flux de données des capteurs en continu et prend l'initiative d'agir lorsqu'il est nécessaire. La comparaison

entre ces deux méthodes met en évidence la nécessité de trouver un compromis entre sécurité, confort et contraintes matérielles.

- Le mode réactif garantit le confort de l'utilisateur en évitant toute pollution sonore. Sur le plan technique, il se montre aussi comme le plus économe en ressources, puisque les briques de recherche sémantique et de calcul vectoriel ne sont sollicitées qu'à la demande de l'utilisateur. En revanche, sa limite principale est qu'il dépend fortement de la mémoire de l'utilisateur : si ce dernier oublie l'existence d'un objet, alors l'assistant reste muet.
- Le mode proactif permet d'ajouter une couche de sécurité active indispensable pour la détection d'obstacles notamment. En revanche, appliqué à tous les objets du quotidien, il provoque une surcharge cognitive ainsi que technique.

Afin de combiner les forces de ces deux approches en limitant les inconvénients, le choix a été fait de s'orienter vers une solution hybride. Par défaut, l'architecture logicielle fonctionne en mode réactif. En revanche, le système bascule sur le mode proactif pour les éléments enregistrés avec un tag de priorité critique (priorité 1). Ce compromis permet de garder un certain confort utilisateur tout en conservant l'aspect sécurité si important.

IV. ARCHITECTURE ET IMPLEMENTATION TECHNIQUE

4.1 Introduction au pipeline RAG local

Le système de mémoire développé au cours de ce stage repose sur une architecture hybride de Génération Augmentée par Récupération (Retrieval-Augmented Generation – RAG) conçue pour s'exécuter en local.

Contrairement aux systèmes RAG conventionnels, le pipeline de VisIA doit relever une problématique : celle de traiter un flux continu (capteurs vidéo), et de l'aligner avec des requêtes naturelles (commandes vocales ou textuelles).

La mission qui m'a été confiée concerne la conception et l'optimisation de la brique mémorielle de VisIA. Afin de valider l'architecture sans entrer dans la partie « détection d'objets » qui fait l'objet d'un autre sujet de stage au sein de l'ISIA Lab, l'ensemble des entrées sensorielles est simulé via une interface développée sous Gradio (le contexte visuel est simulé sous forme de coordonnées horizontales normalisées).

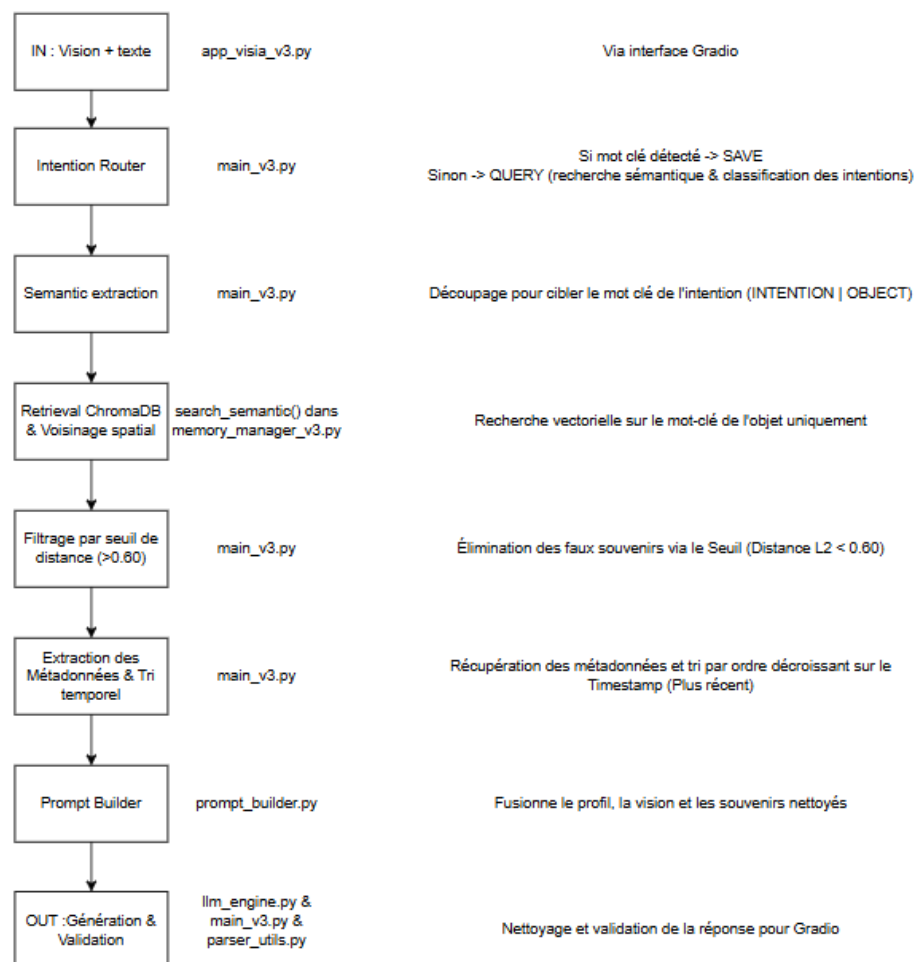


Figure 6 : Schéma du flux de l'architecture du système de mémoire VisIA V3

4.2 Évolution du projet : de la V1 à la V5

Le développement du système de mémoire a connu plusieurs versions, dont l'évolution conduit aux versions de 1 à 5 détaillées ci-après. Le code complet est disponible en annexe ([voir Annexe A > Dépôt du projet VisIA](#)).

4.2.1 RAG sémantique naïf (V1)

La première version consistait en un pipeline RAG classique. Les requêtes de l'utilisateur et les détections simulées des capteurs étaient envoyées à un modèle d'embeddings, puis stockées de manière brute au format JSON dans une liste.

Pour l'évaluation du système, un benchmark initial a été conçu afin de tester la persistance et la fidélité de la restitution des souvenirs. Cette approche a montré des limites avec un taux de succès de seulement 14%. Cet échec a été causé par deux problèmes majeurs :

- Le modèle de langage recevait les souvenirs dans le désordre, il n'avait pas de notion de temps et confondait présent, passé et futur.
- Le modèle léger Gemma 3 (4B) échouait fréquemment à respecter les structures de données JSON imposées par les frameworks de mémoire (ici Mem0). Le moindre caractère manquant provoquait instantanément un échec dans la réponse.

4.2.2 Approche par fichiers (V2)

Pour corriger la V1, la version 2 a introduit trois briques issues de l'étude de l'état de l'art : MaT, TSM, ainsi qu'un radar spatial mesurant la distance mathématique entre le regard (simulé) et les objets de la scène.

Cette architecture reposait sur l'ouverture et la lecture de fichiers textes (au format JSON), et bien que le taux de succès soit monté à 42%, cette méthode s'est révélée trop lourde et générait une latence importante dès lors que ces fichiers devenaient eux-mêmes plus volumineux.

4.2.3 Centralisation sous ChromaDB et décroissance temporelle (V3)

La version 3 a marqué une rupture, les fichiers au format JSON ont été remplacés par le vector store de ChromaDB. Grâce à ce changement, la latence de recherche a fortement diminué (inférieure à 0.5 s). Ensuite, pour résoudre le conflit de temporalité, un algorithme de décroissance temporelle (Temporal Decay) a été développé. Ce dernier est inspiré de la théorie de la courbe de l'oubli d'Ebbinghaus (illustrée en Figure 7), qui modélise la façon dont la mémoire humaine diminue avec le temps si elle n'est pas rafraîchie. Ainsi, dans notre architecture, plus un souvenir est ancien, plus sa distance mathématique est pénalisée. Le système introduit également des bonus de fraîcheur pour les souvenirs enregistrés récemment (inférieurs à 2 min). Enfin, le routeur d'intentions a été corrigé grâce à un format textuel attendu strict (INTENTION | OBJET_CORRIGE). Le taux de succès obtenu est de 76%.

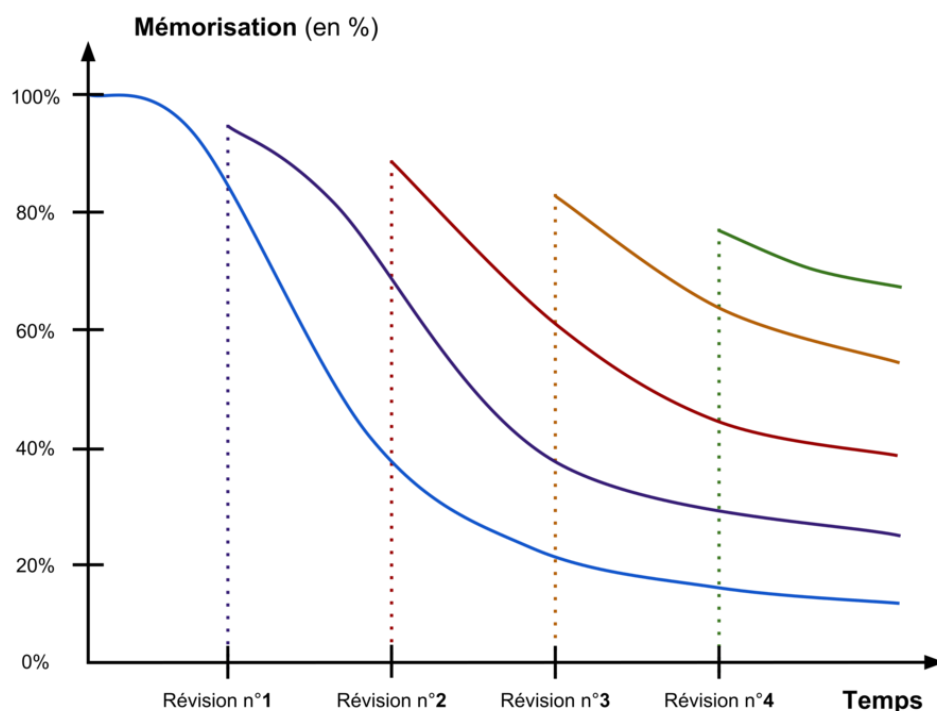


Figure 7 : Courbe de l'oubli de Hermann Ebbinghaus

4.2.4 Intégration du graphe sémantique (V4)

Malgré les performances améliorées de la V3, un problème persistait : si l'utilisateur déplaçait un objet d'un point A à un point B, la base vectorielle conservait les deux mentions en mémoire. En interrogeant le système par la phrase « Quels objets sont proches de mes clés », la recherche vectorielle fusionnait les multiples anciens souvenirs (position actuelle avec anciennes positions). Pour y remédier, l'architecture V4 intègre un graphe sémantique où chaque objet devient un « nœud » et les objets co-présents forment des « arcs directionnels ». Pour garantir la cohérence entre le passé et le présent, un algorithme de rupture d'arc a été implémenté. À chaque nouvel enregistrement, le script interroge ChromaDB pour identifier les liens précédents de l'objet concerné. Les relations de proximité devenues obsolètes sont actualisées, tandis qu'un nouveau nœud est créé et rattaché aux objets de la scène actuelle. Cette approche a été évaluée lors d'une campagne de test sur des scénarios de déplacement et a atteint un taux de réussite de 100% sur ces cas d'usages spécifiques.

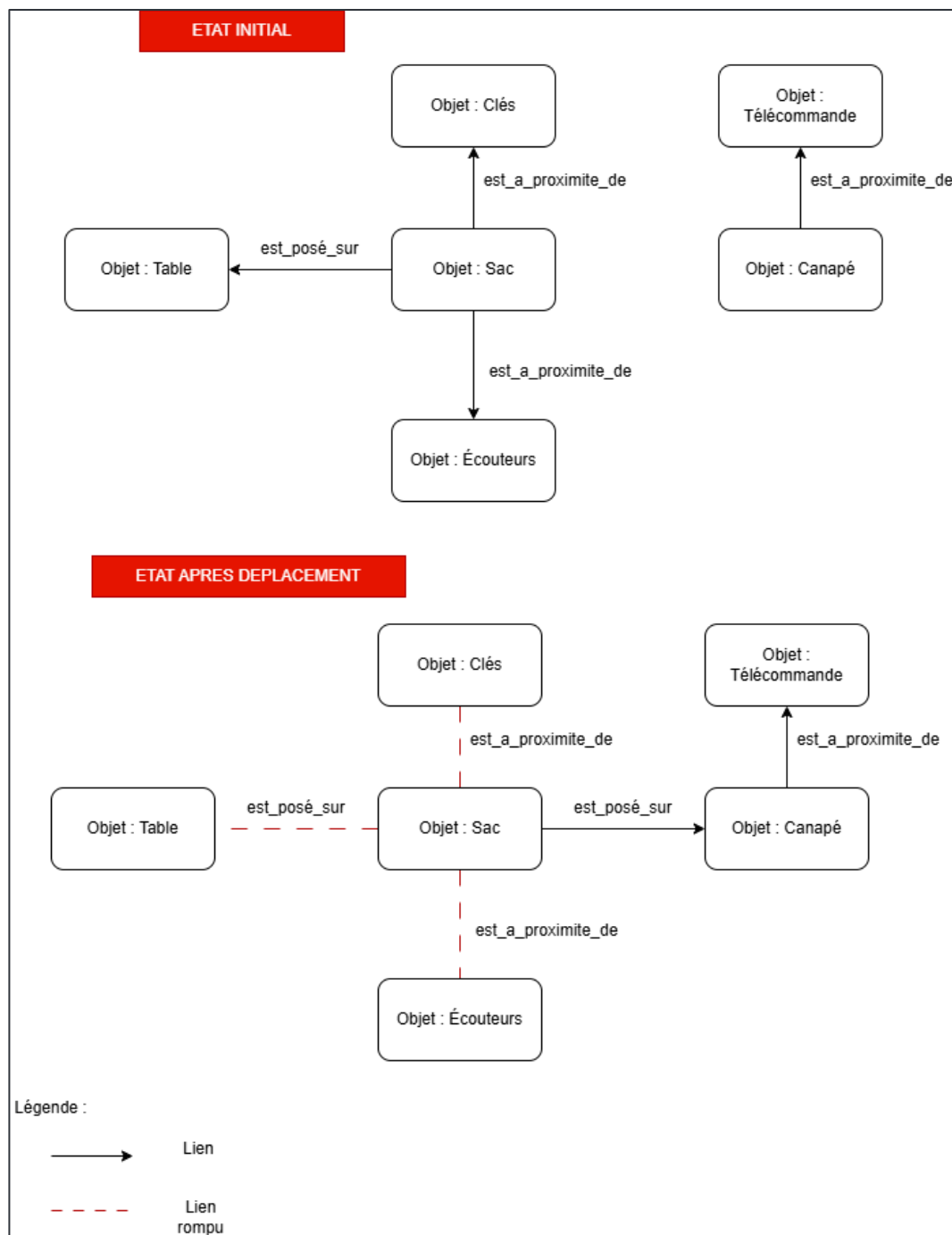


Figure 8 : Schéma du principe de rupture d'arc lors d'un changement dans la scène

4.2.5 Architecture hybride et optimisation du routage (V5)

Afin de consolider la V4, la version 5 introduit une architecture hybride ajoutant une nouvelle brique de mémoire à long terme (Mem0). Cette brique gère spécifiquement les habitudes et préférences de l'utilisateur. Cette version revisite aussi l'utilisation du moteur d'inférence, en effet, il a été observé que la phase de classification des intentions représentait une source latence pour le système car le modèle Gemma 3 (4B) comporte un nombre important de paramètres. Le choix a été fait de passer à une utilisation d'un modèle léger dédié à la classification des intentions (modèle Llama-3.2-1B). Gemma 3 (4B) reste le modèle

utilisé pour la génération de la réponse finale tandis que le modèle léger se charge exclusivement du routage.

4.3 Spécifications des Modules

4.3.1 Classification et routage d'intentions (Intention Router)

Le bloc à l'entrée du pipeline est le routeur d'intention. Son rôle est d'aiguiller la requête en se basant sur la sémantique. Il analyse la phrase formulée par l'utilisateur et en déduit l'action requise tout en isolant l'objet cible.

Afin de fiabiliser ce module sans augmenter la latence, une stratégie de Few-Shot Prompting a été utilisée (Ajout de quelques exemples dans le prompt pour guider le modèle de langage dans sa réponse). Le choix a aussi été fait d'utiliser une syntaxe simple car les modèles légers rencontrent des difficultés à générer une réponse suivant une syntaxe stricte et complexe.

Structure du prompt d'aiguillage :

Le modèle reçoit un prompt lui interdisant tout bavardage, cela permet d'éviter de longues réponses complexes car dans notre cas d'assistance à personnes déficientes visuelles des réponses courtes et précises sont souhaitées. Ce même prompt impose au modèle de classer la requête parmi sept intentions définies ci-après, et de retourner le résultat sous la forme syntaxique suivante : INTENTION | OBJET_CORRIGÉ.

- **SAVE** : L'utilisateur indique qu'il pose, range ou déplace un objet. Le système enregistre l'action.
- **SAVE_USER_HABIT** : L'utilisateur indique quelles sont ses habitudes et définit son profil d'utilisateur. Le système enregistre directement ces informations dans la brique de profil Mem0.
- **get_user_habit** : L'utilisateur pose une question sur ses habitudes, ses routines ou son profil. Le système interroge la brique Mem0.
- **find_object_by_category** : L'utilisateur cherche la position actuelle d'un objet. Le système se réfère à ses capteurs de vision ou à ses souvenirs pour retrouver la position de l'objet.
- **find_past_location** : L'utilisateur cherche la position passée d'un objet. Le système se réfère uniquement à ses souvenirs pour retrouver la position antérieure de l'objet.
- **find_critical_objects** : L'utilisateur cherche à lister les objets sous surveillance active (priorité 1).
- **none** : Intention définie par défaut lorsque la requête de l'utilisateur est hors-scope, ce qui déclenche un message de sécurité. Cela permet de bloquer les hallucinations du modèle de langage qui a tendance à inventer lorsqu'il ne connaît pas la réponse à la requête de l'utilisateur. Parmi ces requêtes hors-scope on retrouve les questions de culture générale, les calculs, les messages de salutations, etc.

Une fois la chaîne de caractères renvoyée par le LLM (ex. : `find_object_by_category | badge`), un traitement de nettoyage est appliqué. Autrement dit, plutôt que de lancer une recherche vectorielle sur la phrase complète de l'utilisateur (ex. : `Dis-moi, je ne sais plus où j'ai mis mon portefeuille, ça fait plusieurs heures que je le cherche`), ChromaDB est interrogée uniquement sur le mot principal (ex. : `portefeuille`). Ce qui augmente la précision lors de la phase de récupération.

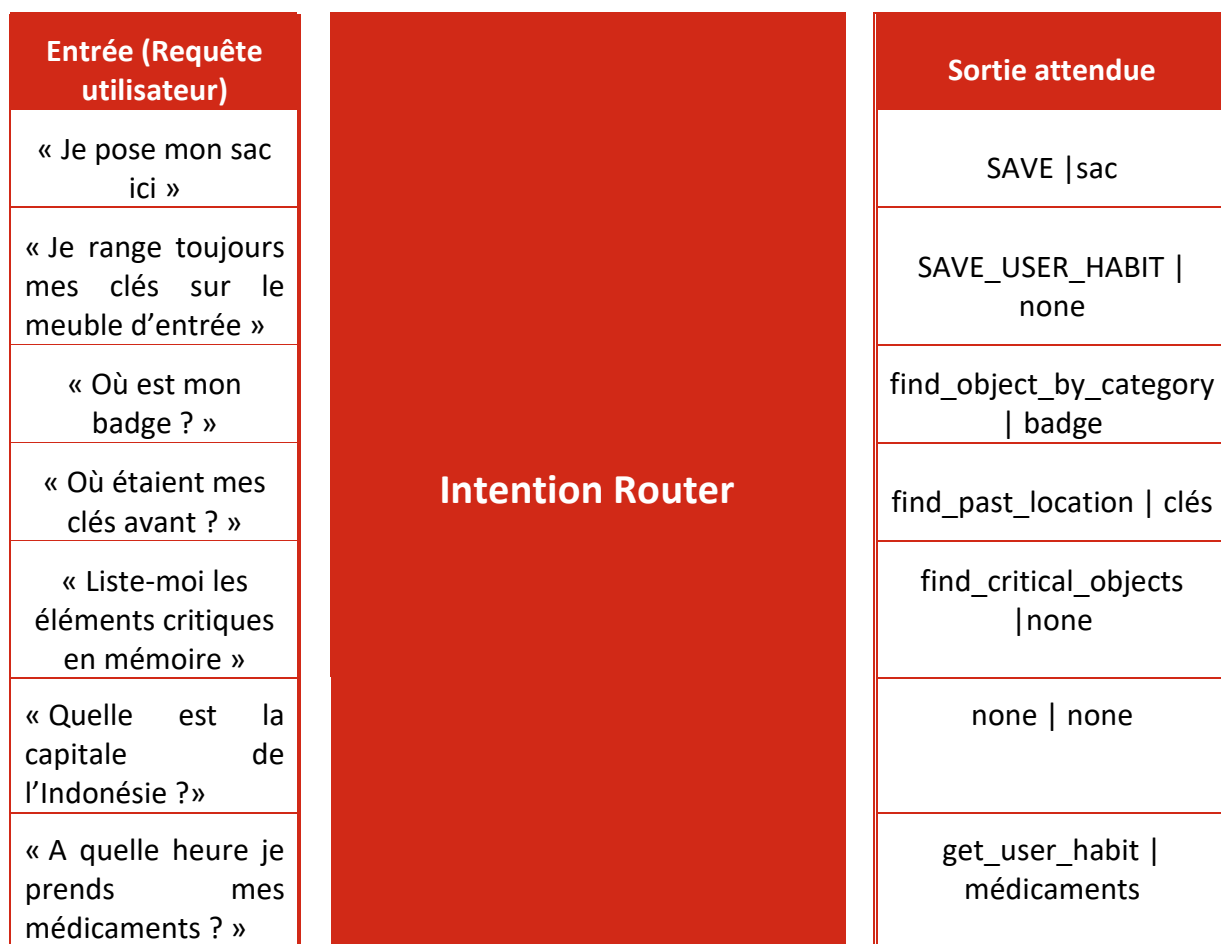


Figure 9 : Exemple de routage de l'Intention Router

4.3.2 Gestion des coréférences et traitement de la syntaxe française

La langue française présente des ambiguïtés grammaticales qui peuvent être compliquées à comprendre pour un système informatique. Pour cela, il existe des outils de traitement des langues naturelles. Le domaine du NLP (Natural Language Processing, ou Traitement Automatique des Langues) regroupe les méthodes permettant à une machine de comprendre et interpréter le langage humain. Deux problèmes linguistiques majeurs ont été traités au cours du stage : la résolution des coréférences et la normalisation sémantique.

Résolution des coréférences :

Dans un contexte de discussion continue, l'utilisateur a souvent tendance à utiliser des pronoms pour désigner un objet mentionné précédemment (ex. : « Où sont mes clés ? » suivi de « Où sont-elles ? »). Un RAG classique chercherait le mot « elles » dans sa base de données, ce qui provoquerait un échec car le mot « elles » est éloigné sémantiquement du mot « clés ».

Bien qu'il existe des bibliothèques de calcul spécialisées en NLP (telles que spaCy avec son modèle fr_core_news_sm testé au cours du stage), celles-ci ont été écartées de l'architecture finale. En effet, les phases de tests ont mis en évidence que ces modèles légers sont inadaptés pour le français. C'est pourquoi une approche par prompt a été préférée : le système s'appuie sur le LLM en lui imposant, via un prompt, d'utiliser l'historique récent de la session. Ainsi, le LLM remplace automatiquement le pronom par le nom de l'objet cible détecté précédemment. Les tests de validation de cette méthode montrent des taux de

réussite élevés, notamment sur les pronoms génériques et les contextes complexes, comme l'illustre la figure ci-dessous.

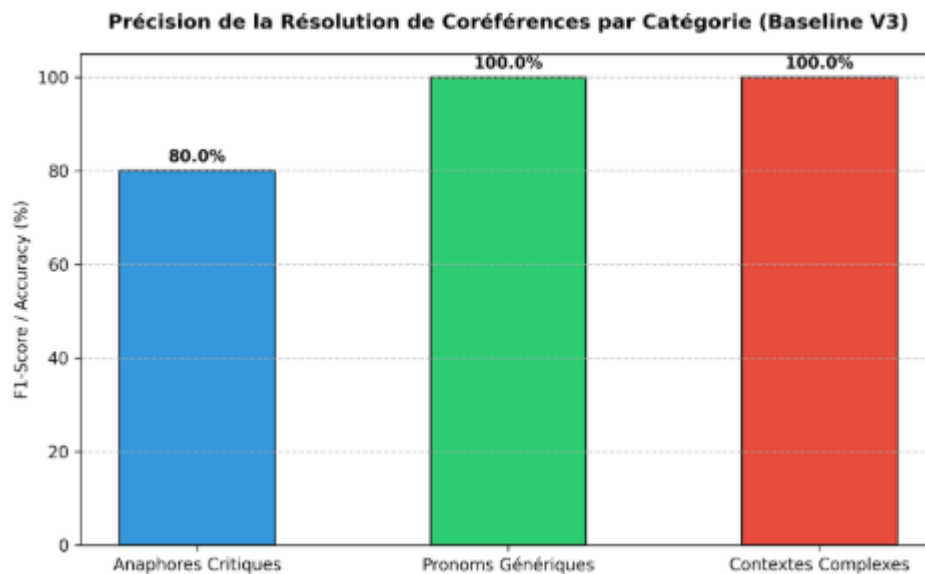


Figure 10 : Graphe de précision de la résolution de coréférences par catégorie de test

Normalisation sémantique (Lemmatisation) :

Une fois l'objet extrait de la requête par le routeur d'intentions, une seconde étape de traitement linguistique intervient. Ce module prend en charge la normalisation sémantique et la lemmatisation des entités. La lemmatisation consiste à ramener un mot sous sa forme « neutre », c'est-à-dire sa forme singulière pour les noms et son infinitif pour les verbes. Cette opération est essentielle pour harmoniser le stockage. Si l'utilisateur enregistre un souvenir en disant « je pose mes clés » (pluriel) et demande plus tard « Où est ma clé ? » (singulier), la base vectorielle fait face à deux chaînes de caractères différentes. Malgré des embeddings proches, la précision du calcul de distance peut être impactée. Pour standardiser les requêtes, le script Python nettoie l'objet extrait :

- Passage en minuscule (.lower())
- Nettoyage des espaces (.strip())
- Extraction des lemmes.

V. VALIDATION ET ANALYSE DES RESULTATS

5.1 Protocoles de test et métriques d'évaluation

5.1.1 Analyse des benchmarks existants

Lors des premières phases d'évaluation du système, j'ai eu l'occasion d'étudier plusieurs benchmarks de référence en matière de mémoire conversationnelle (LongMemVal, MemoryBench, PersonaMem-v2, MemoryAgentBench, FactConsolidation). L'objectif était de savoir s'ils pouvaient être utilisés afin de mesurer la capacité du modèle à retenir et restituer des souvenirs sur de longues sessions. Cependant, l'application de ces datasets à l'architecture de VisIA a mis en évidence plusieurs limites :

- Les benchmarks existants ne prennent pas en compte la dimension multimodale (spatiale + temps réel) essentielle à l'assistance de personnes déficientes visuelles.
- Des contraintes lors de l'exécution en local dues à des contextes trop longs (LongMemVal = 32k tokens, PersonaMem-v2 = 128k tokens) ont provoqué des timeouts récurrents.
- Les LLM ont tendance à avoir des difficultés à supprimer ou écraser une information en mémoire sans un mécanisme de filtrage à côté.

5.1.2 Enseignements tirés des benchmarks existants

Face aux limites de ces benchmarks existants, non adaptés à VisIA, le choix a été fait de concevoir un protocole de test sur mesure (50 tests), reprenant essentiellement les cas d'usage visés et pensés lors de la définition des use cases (sauvegarde de position, recherche d'objets, gestion des habitudes, alertes de sécurité, requêtes hors-scope).

Ce protocole évalue le système selon deux axes principaux :

- La précision du routeur d'intentions, illustrée par la matrice de confusion (figure 11).
- L'évaluation de la latence d'inférence moyenne.

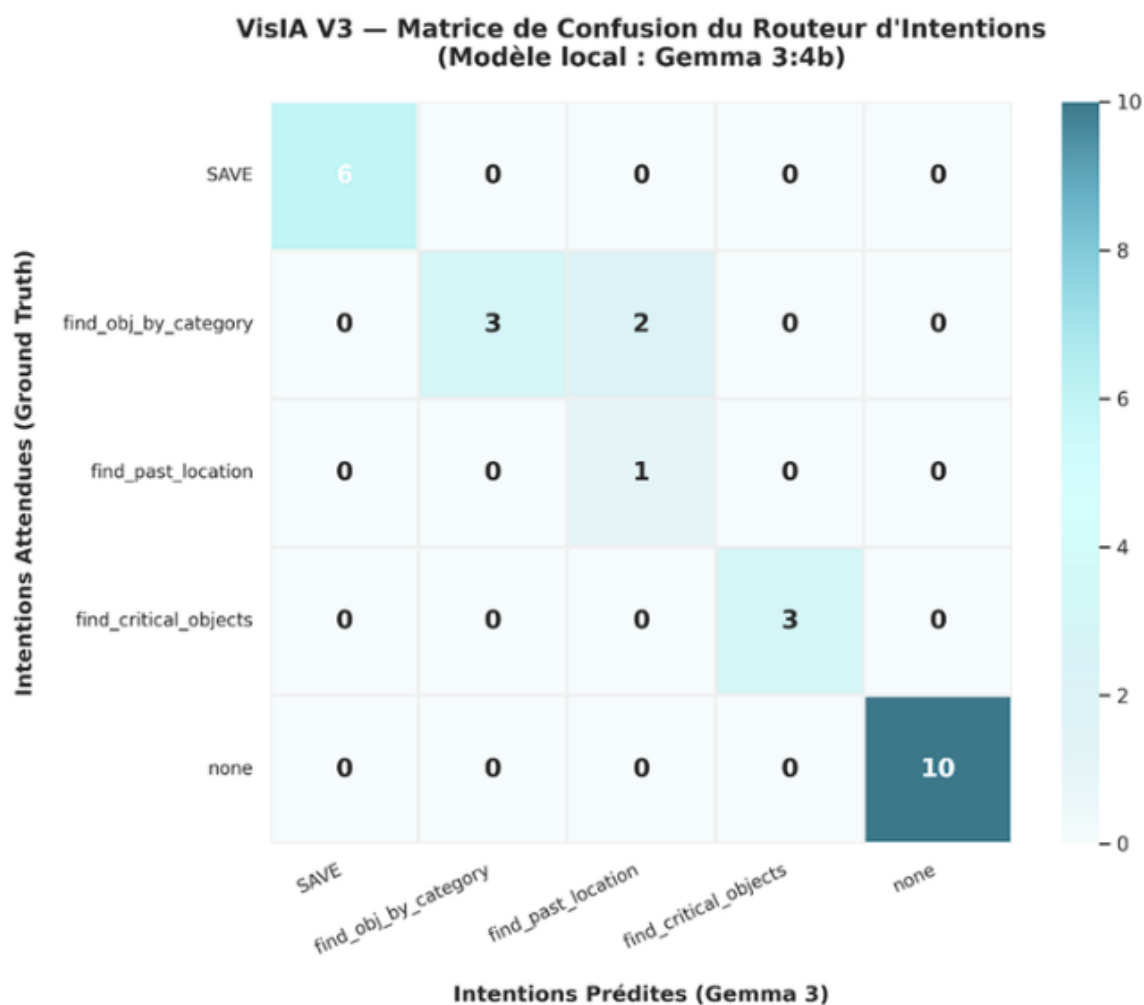


Figure 11 : Matrice de Confusion du Routeur d'Intentions (V3)

5.2 Analyse globale des résultats

5.2.1 Optimisations de l'architecture et gains de performance

Lors des premiers tests, le premier message affichait un temps de réponse de 112.4 secondes à cause du chargement initial des modèles en VRAM (Video Random Access Memory). Pour résoudre cette problématique, une pré-requête est lancée avant la création de l'interface graphique, cela permet de charger les modèles en amont. Grâce à cette optimisation, la latence du premier message a chuté, rendant le système exploitable directement.

Par ailleurs, l'algorithme de décroissance temporelle développé à l'occasion de la V3 permet d'ajuster la distance vectorielle en appliquant un bonus de récence pour les souvenirs récents, et de pénaliser les souvenirs plus anciens (comme le montre la figure 12).

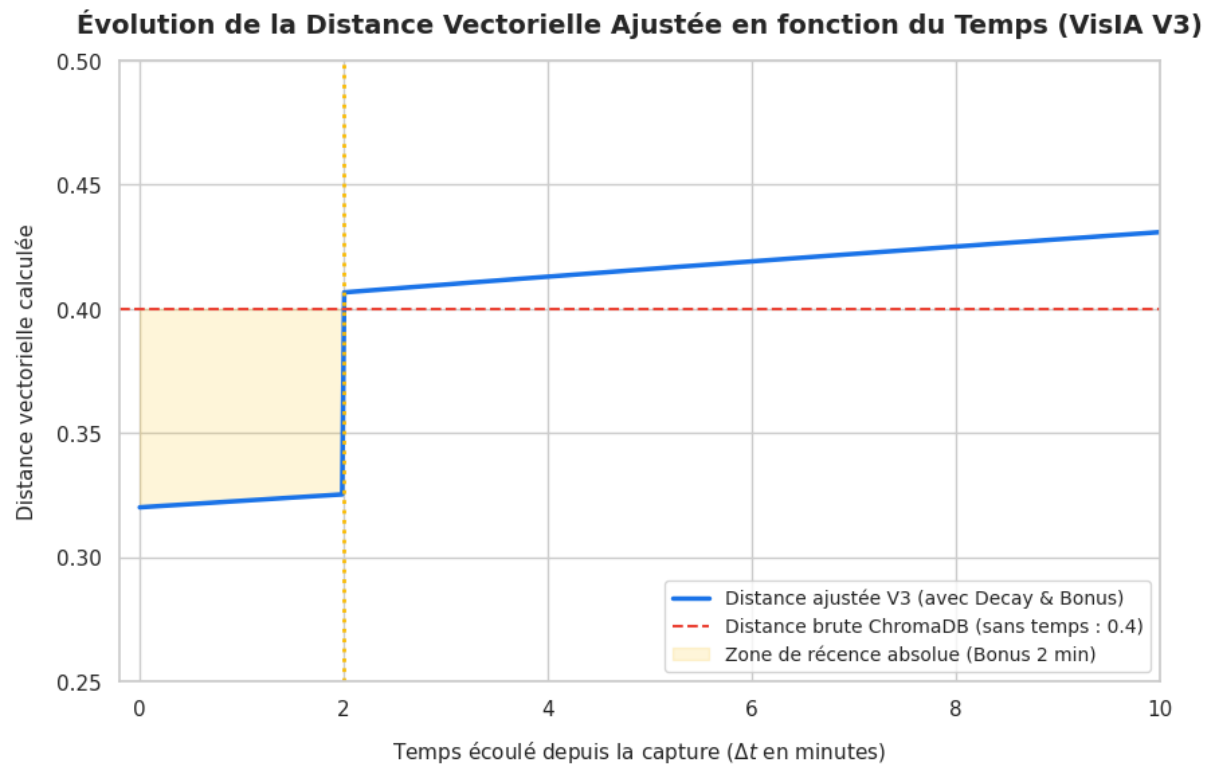


Figure 12 : Évolution de la distance vectorielle ajustée en fonction du temps.

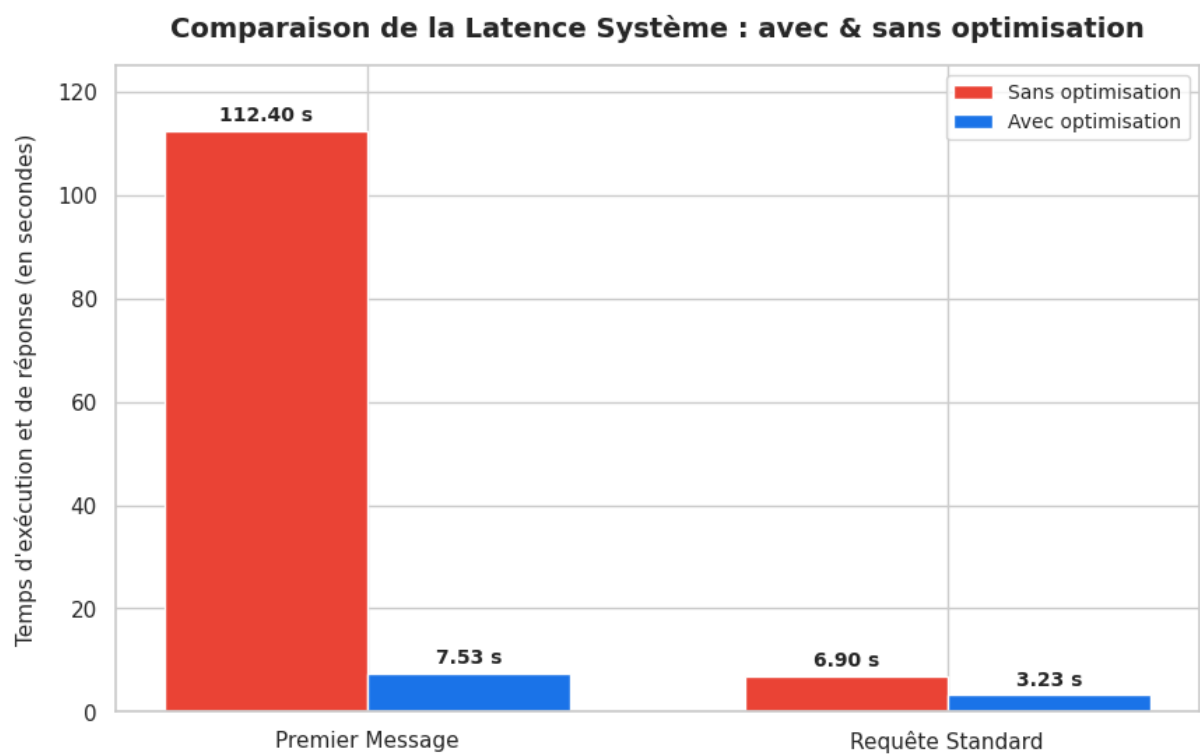


Figure 13 : Comparaison des gains de latence (avec et sans optimisation de préchargement)

5.2.2 Bilan final des performances de la V5

Le benchmark final pour la version 5 teste 50 cas d'usages, il démontre l'efficacité globale de l'architecture.

L'analyse de la précision du routeur par intention, présentée dans la figure 14, prouve d'excellents résultats sur la plupart des intentions, avec notamment un taux de réussite de 100% pour les intentions SAVE, find_past_location, find_critical_object, et none.

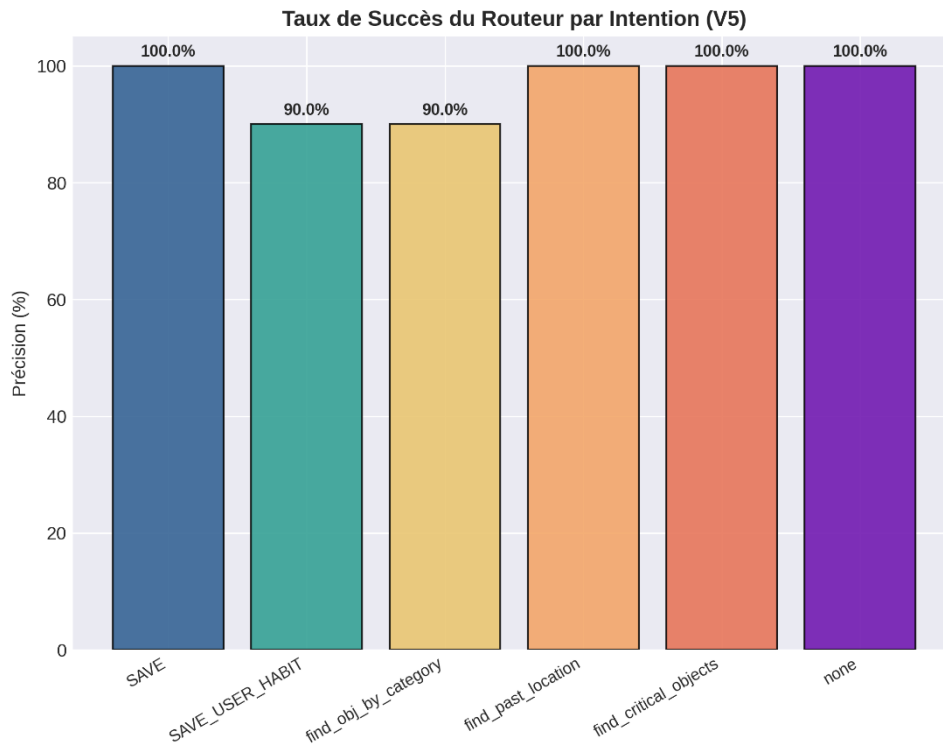


Figure 14 : Taux de succès du routeur par intention (V5)

Au sujet de la réactivité, la distribution de la latence des requêtes (illustrée en figure 15) montre une moyenne à 5.01s par requête. Il est important de souligner le fait que la grande majorité des requêtes s'exécutent entre 3 et 5 secondes. Malgré les différentes optimisations réalisées qui ont permis de réduire ce temps de réponse, il reste toujours des requêtes pour lesquels le temps d'exécution est plus long (en particulier lorsque la requête de l'utilisateur fait appel à ChromaDB en même temps que Mem0).

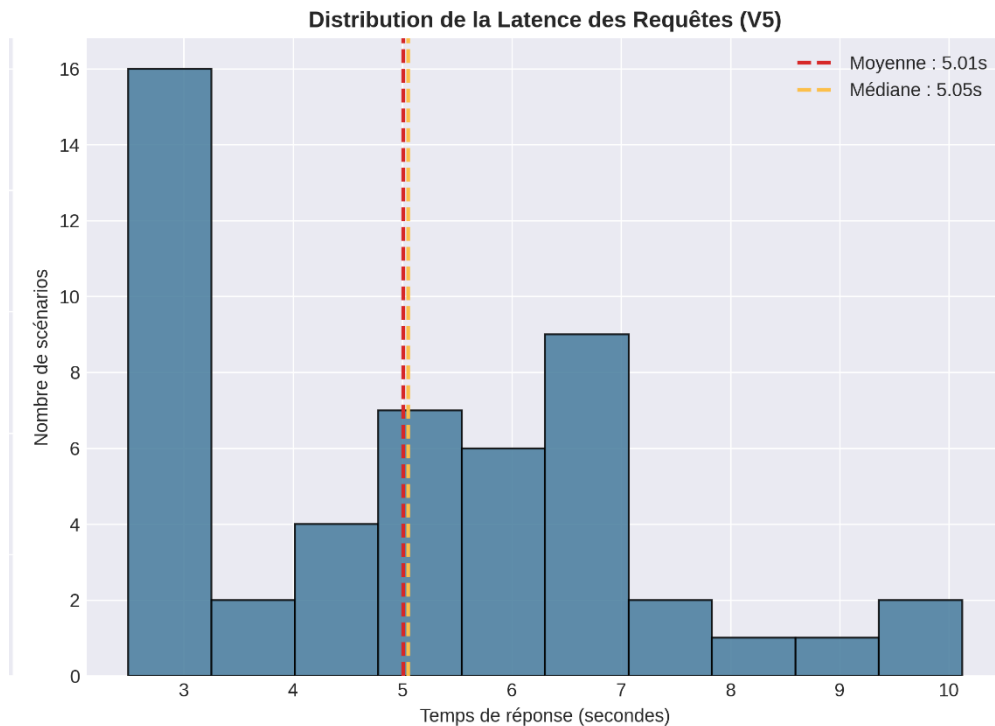


Figure 15 : Distribution de la latence des requêtes (V5)

Enfin, l'analyse du diagramme de la figure 16 met en avant les écarts de performance selon la brique sollicitée. On remarque que la brique d'enregistrement brute (SAVE) s'exécute très rapidement (environ 1.5 s), tandis que la gestion des profils via Mem0 (SAVE_USER_HABIT) demande un temps d'exécution plus important (autour de 7 s).

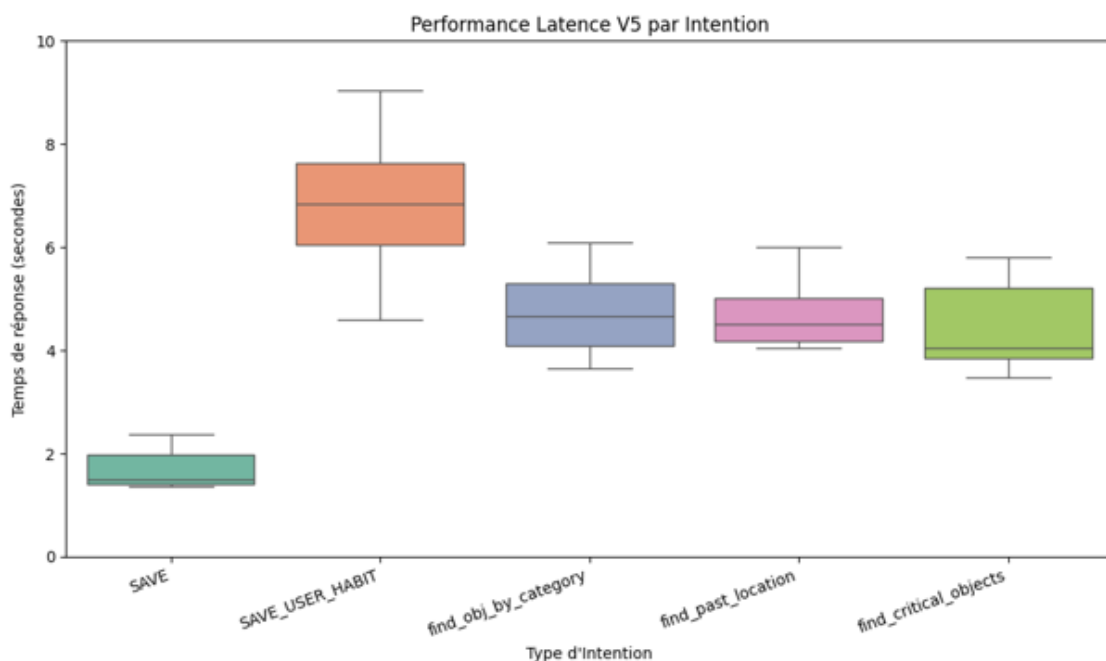


Figure 16 : Diagramme du temps de réponse du système en fonction de l'intention (V5)

VI. CONCLUSION

Ce stage a été une expérience enrichissante pour moi. En effet, j'ai eu l'occasion au cours de ces vingt-deux semaines de travailler en grande autonomie et d'acquérir des compétences techniques et méthodologiques solides. Ce stage a répondu à mes attentes initiales, qui étaient de travailler sur un projet d'envergure liant l'intelligence artificielle à l'humain, en l'occurrence l'assistance aux personnes déficientes visuelles.

J'ai toutefois pu être frustré par moment car la recherche ne permet pas toujours d'avoir une visibilité à long terme sur les objectifs précis et comment y parvenir. Alors que je privilégie d'habitude une méthodologie de travail basée sur une roadmap structurée, cela m'a forcé à travailler dans l'inconfort.

Malgré cela, ce stage m'aura fait évoluer professionnellement et personnellement. En effet, grâce à ce stage, j'ai acquis un regard nouveau sur les enjeux du secteur, sur les méthodes de travail, et cela me sera fortement utile dans la suite de ma carrière. J'ai également eu l'occasion de travailler sur un projet où l'on ne cherche pas uniquement la performance algorithmique, mais qui répond aussi à des exigences humaines, éthiques, et inclusives. J'ai particulièrement apprécié faire de la veille technologique en découvrant l'état de l'art récent.

Concernant le projet VisIA, je suis conscient que mon travail est une base qui nécessite des ajustements avant de pouvoir envisager une mise en production. Néanmoins, j'estime avoir construit des fondations qui permettront par la suite au projet de se développer. Enfin, pour le futur, la perspective logique consiste à coupler cette architecture mémorielle avec des flux vidéo issus de caméras pour se rapprocher de l'usage réel.

VII. ANNEXES

7.1 Annexe A : Liens et références des outils et frameworks :

- Dépôt du projet VisIA : <https://github.com/OmarSeddati/stage-9-Thibault>
- Frameworks de mémoire :
 - mem0ai : [GitHub Docs](#)
 - OpenMemory : [GitHub](#)
 - Cognee : [GitHub Docs](#)
 - Letta/MemGPT : [GitHub Docs](#)
 - Zep : [GitHub Docs](#)
- Moteur d'inférence et bases de données vectorielles :
 - Ollama : [Docs](#)
 - ChromaDB : [Documentation officielle](#)
- Outils d'infrastructure et d'interface :
 - Langchain : [GitHub Docs](#)
 - Motorhead : [GitHub](#)
 - NeuroMemory : [GitHub](#)
 - CrewAI : [GitHub Docs](#)
 - LlamaIndex : [GitHub Docs](#)
- Modèles de langage et d'embeddings :
 - https://ai.google.dev/gemma/docs/core/model_card_3?hl=fr
 - <https://ai.google.dev/gemma/docs/core?hl=fr>
 - <https://docs.nomic.ai/reference/api/embed-text-v-1-embedding-text-post>
 - <https://huggingface.co/nomic-ai/nomic-bert-2048>
- Traitement de la langue et Outils NLP :
 - <https://spacy.io/api/doc>
 - <https://spacy.io/models/fr>
 - [https://fr.wikipedia.org/wiki/Courbe de l'oubli](https://fr.wikipedia.org/wiki/Courbe_de_l'oubli)
 - [https://fr.wikipedia.org/wiki/Okapi BM25](https://fr.wikipedia.org/wiki/Okapi_BM25)
- Projets similaires, architectures open-source et intégrations :
 - LangGraph & Mem0 : <https://github.com/dorianlgs/langgraph-mem0>
 - SidekickOS : <https://github.com/siersidekick/SidekickOS>

- Chatbots Ollama & Gradio / Chainlit :
 - <https://github.com/mirec84/ollama-chatbot-gradio/>
 - [https://github.com/SACTAN/RAG with Ollama Gradio](https://github.com/SACTAN/RAG_with_Ollama_Gradio)
 - <https://github.com/fanqingsong/rag-ollama-langchain>
 - <https://github.com/sudarshan-koirala/langchain-gemma-ollama-chainlit/blob/main/README.md>
- Syrin Harness : <https://github.com/syrin-labs/syrin-harness>
- Références et articles scientifiques étudiées :
 - Repo GitHub contenant des articles state of the art : <https://github.com/VoltAgent/awesome-ai-agent-papers>
 - Beyond Dialogue Time: Temporal Semantic Memory for Personalized LLM Agents (Su et al.) : <https://arxiv.org/abs/2601.07468>
 - Enhancing Coreference Resolution with Pretrained Language Models: Bridging the Gap Between Syntax and Semantics (Liu et al.) : <https://arxiv.org/abs/2504.05855>
 - FadeMem: Biologically-Inspired Forgetting for Efficient Agent Memory (Wei et al.) : <https://arxiv.org/abs/2601.18642>
 - FastInsight: Fast and Insightful Retrieval via Fusion Operators for Graph RAG (An et al.) : <https://arxiv.org/abs/2601.18579>
 - MemCtrl: Using MLLMs as Active Memory Controllers on Embodied Agents (Dorbola & Manocha) : <https://arxiv.org/abs/2601.20831>
 - Spatial-RAG: Spatial Retrieval Augmented Generation for Real-World Geospatial Reasoning Questions (Yu et al.) : <https://arxiv.org/abs/2502.18470>
 - Utilizing Metadata for Better Retrieval-Augmented Generation (Yousuf et al.) : <https://arxiv.org/abs/2601.11863>

7.2 Annexe B : Veille technologique et solutions existantes sur le marché

- Be My Eyes : <https://www.bemyeyes.com/>
- Microsoft Seeing AI : <https://www.microsoft.com/en-us/garage/wall-of-fame/seeing-ai/>
- Google Lookout : <https://www.lookout.com/tags/google>
- Oorion : <https://www.oorion.fr/>
- OrCam My Eye : https://www.orcham.com/fr-fr/home?srsId=AfmBOor-W_DVzRORGGBevEygwKLiCt9JiZV_lk0TWksrwdu4CXROe2Ea
- Envision glasses : <https://www.letsenvision.com/glasses/home>
- Meta rayban : https://www.meta.com/be/fr/ai-glasses/?utm_source=gg&utm_medium=ps&utm_campaign=22556853287&utm_term=meta%20rayban&utm_content=774387352739&utm_ad=179184474629&utm_location=9197254&utm_location2&utm_placement=kwd-1539835765136&utm_device=c&utm_matchtype=b&utm_feed&utm_adposition&utm_product&ads_rl=9213737374&&gclid=aw.ds&gad_source=1&gad_campaignid=22556853287&gbraid=0AAAAAo_xvTmN8JeVWpH1O1h1Ac50kw1l&gclid=Cj0KCQjwrs7RBhDuARIsAIVfBD3365w4cqwwOQppXtX8aGDvTWD71WDpJH1oXr-hmnHF9ZQlrxWcQwaAoXFEALw_wcB

- Angel Eye : <https://www.angeleyeglobal.com/>
- Sidekick : <https://www.sidekicklabs.co/>
- Agiga Echovision : <https://agiga.ai/>

▪ Illustrations du matériel :



Figure 17 : Boîtier OrCam MyEye fixé sur monture



Figure 18 : Lunettes connectées Envision glasses (Ally)



Figure 19 : Lunettes connectées Meta Rayban



Figure 20 : Boîtier AngelEye



Figure 21 : Boîtier Sidekick fixé sur monture



Figure 22 : Lunettes connectées Agiga Echovision

- Ressources complémentaires :
 - Reddit r/Blind : <https://www.reddit.com/r/Blind/>
 - TrustWorthy AI Labs : <https://sites.google.com/view/mlpa/mainpage>
 - ISIA Lab : <https://web.umons.ac.be/isia/>
 - numediart : <https://web.umons.ac.be/numediart/fr/>
 - inforTech : <https://web.umons.ac.be/infortech/fr/>